**Manual Técnico: Backend para Gestión de Estudiantes y Cursos**

**1. Introducción**

Este manual describe la implementación de un backend para gestionar estudiantes, cursos y sus relaciones en un sistema educativo. El backend está desarrollado en **Node.js** y se conecta a una base de datos **MySQL**. A continuación se detallan los endpoints implementados y la lógica del servidor.

**2. Tecnología y Herramientas Utilizadas**

- **Backend**: **Node.js** con **Express.js**
- **Base de Datos**: **MySQL**
- **Paquete de conexión**: **mysql2**
- **Middleware**: **body-parser** para manejar solicitudes JSON.
- **Herramienta de prueba**: **Postman**

3. Endpoints Implementados

**3.1 GET /students**

Este endpoint permite consultar todos los estudiantes registrados en la base de datos.

**Descripción:**

- Recupera todos los estudiantes de la tabla estudiante de la base de datos MySQL.

**Método HTTP:**

- **GET**

**Ruta:**

http://localhost:3000/students

**Respuesta Exitosa (200)**:


```
[
  {
    "id_estudiante": 1,
```

"nombre": "Juan Pérez",
"carnet": "2023-12345",
"correo": "juan.perez@example.com"
},
{
"id_estudiante": 2,
"nombre": "Ana Gómez",
"carnet": "2023-67890",
"correo": "ana.gomez@example.com"
}
]

**Respuestas de Error:**

- **404**: No se encontraron estudiantes.
- **500**: Error en la base de datos.

### 3.2 POST /register

Este endpoint permite registrar un nuevo estudiante en la base de datos.

**Descripción:**

- Inserta un nuevo estudiante en la tabla estudiante con los datos proporcionados en la solicitud.

**Método HTTP:**

- **POST**

**Ruta:**

**http://localhost:3000/register**

**Cuerpo de la Solicitud (JSON):**

```
{
    "nombre": "Carlos López",
    "carnet": "2023-11223",
    "correo": "carlos.lopez@example.com"
}
```

**Respuesta Exitosa (201)**:

```
{
    "message": "Estudiante registrado exitosamente",
    "studentId": 3
}
```

**Respuestas de Error:**

- **400**: Datos incompletos. Todos los campos son requeridos.
- **500**: Error en la base de datos.

**4. Lógica Implementada**

**Conexión a la Base de Datos**

El servidor utiliza un pool de conexiones **MySQL** para interactuar con la base de datos. La configuración de la conexión se realiza utilizando el paquete **mysql2/promise** para soportar **async/await**.

**Conexión al Pool:**

```
const mysql = require('mysql2/promise');


const pool = mysql.createPool({
    host: 'localhost',
```

```
    user: 'root',
    password: 'tu_contraseña',
    database: 'mi_base_de_datos',
    waitForConnections: true,
    connectionLimit: 10,
    queueLimit: 0
});
```

**Consultas a la Base de Datos:**

La consulta para obtener todos los estudiantes es la siguiente:

```
const [rows] = await pool.query('SELECT * FROM estudiante');
```

**Inserción de Datos en la Base de Datos:**

El registro de un estudiante se realiza con la siguiente consulta:

```
const query = 'INSERT INTO estudiante (nombre, carnet, correo) VALUES (?, ?, ?)';
await pool.query(query, [nombre, carnet, correo]);
```

**5. Instalación y Configuración**

Requisitos Previos:

- Node.js instalado en el sistema.
- MySQL en ejecución en localhost (puerto 3306 por defecto).
- Postman para probar los endpoints.

Pasos para Ejecutar el Proyecto:

1. Clonar el repositorio:
    - Ejecuta git clone <repositorio_url> para clonar el proyecto.
2. Instalar las dependencias:

- Navega al directorio del proyecto y ejecuta:

  npm install

3. Configurar la Base de Datos:
   - Crea una base de datos llamada mi_base_de_datos en MySQL.
   - Ejecuta los scripts SQL proporcionados en el archivo create_tables.sql para crear las tablas necesarias.
4. Configurar el archivo de conexión:
   - Asegúrate de que las credenciales de la base de datos estén correctas en el archivo de configuración de la conexión.
5. Iniciar el servidor:
   - Ejecuta:

     node index.js
6. Probar los endpoints:
   - Usa Postman para probar los endpoints GET /students y POST /register.

**6. Conclusión**

Este backend proporciona una interfaz sencilla para gestionar estudiantes, cursos y sus interacciones. Utiliza **MySQL** como base de datos, y **Node.js** con **Express.js** como servidor. Los endpoints permiten insertar nuevos registros y consultar datos existentes desde la base de datos.

## Anexos

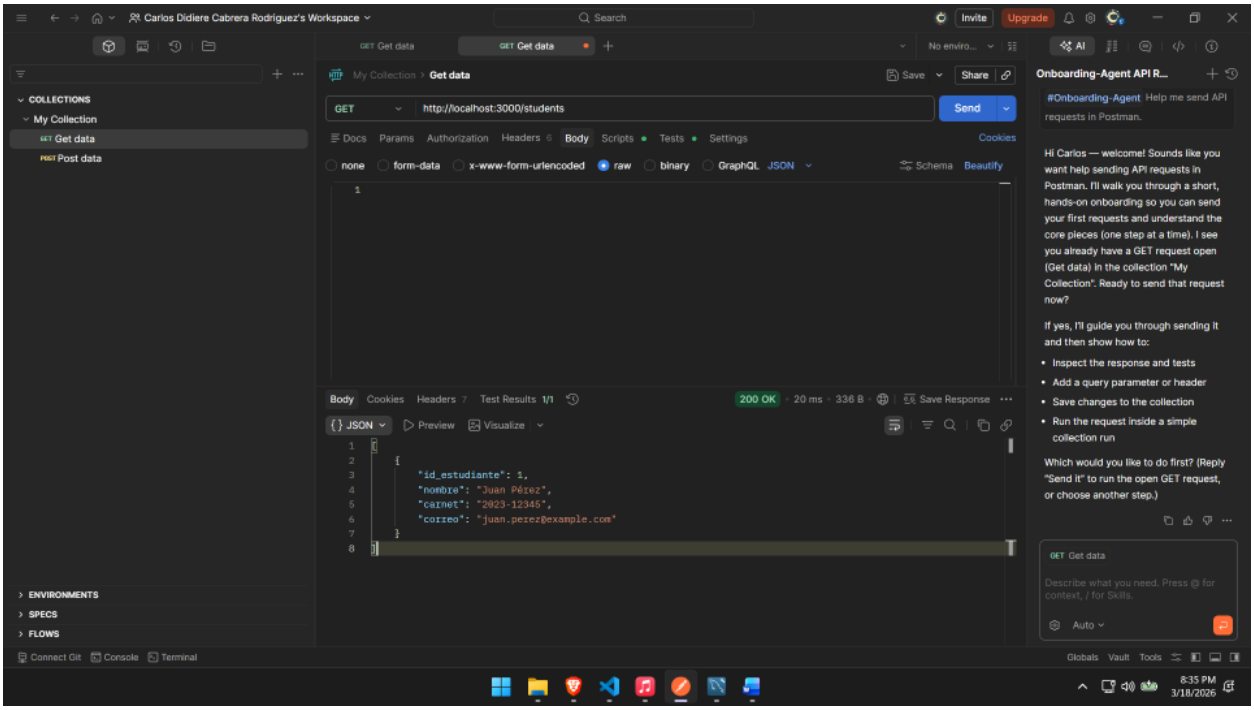


Imagen en la que se visualiza el resultado al conseguir la ruta "http://localhost:3000/students".

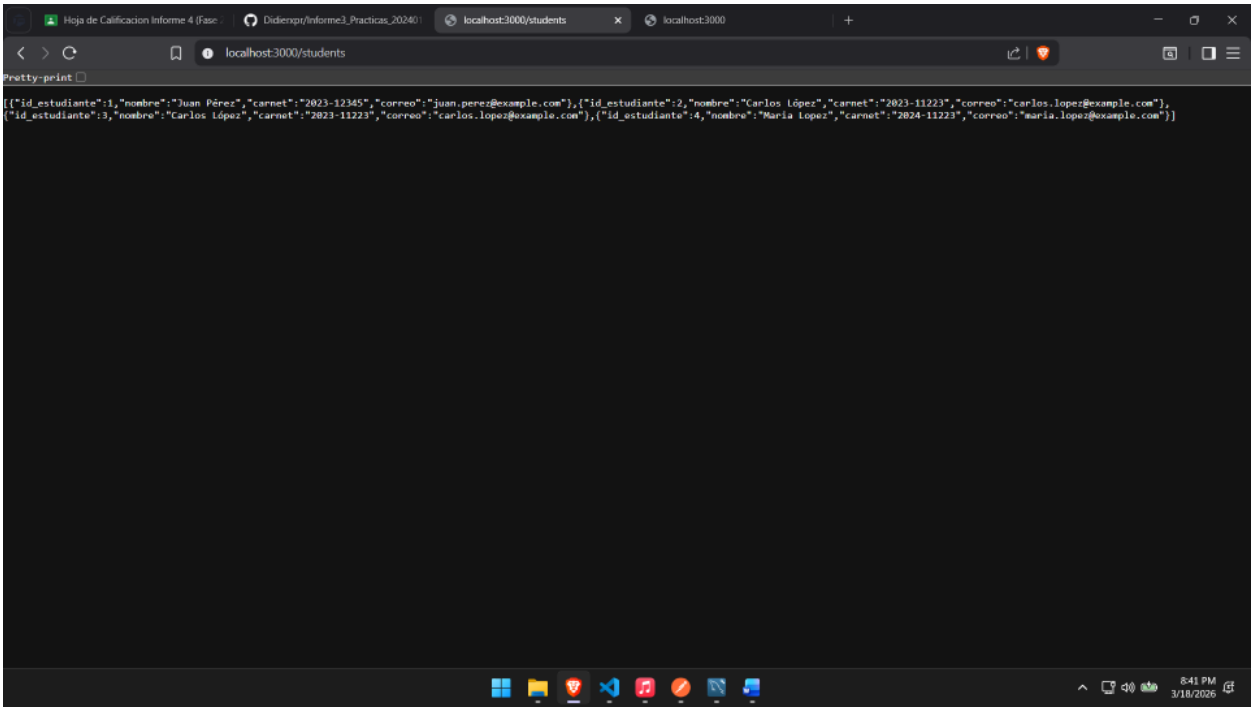


Imagen en la que se visualiza el resultado en el navegador.